Cambios al proyecto - Proyecto 3

Fergye Zhamyra Ortiz Gómez - 202426021

Juan Manuel Forero Bohorquez - 2026514666

Partimos de nuestra propia solución del Proyecto 2. No usamos la solución de otros estudiantes, así que todos los cambios de este documento son respecto a nuestro código de la Parte 2.

Cambios respecto a la Parte 2

Punto del enunciado	**Qué cambiamos**	**Archivos**
1. Parser	Se reutiliza parseObjectilang de la Parte 2. Se corrigió la ambigüedad de la resta después de una declaración (sección 3).	Implode.rsc, Plugin.rsc
2. Ejecutar programas.ol	Nuevo intérprete sobre el AST. run(archivo), verifica tipos y, si no hay errores, ejecuta el programa y muestra la salida en la consola. El plugin reconoce *. ol (y sigue aceptando *.obj).	Eval.rsc y Main.rsc (nuevos), Plugin.rsc
3. Instalar TypePal	Dependencia org. rascalmpl: typepal:0.17.4 en pom.xml. El plugin muestra los errores de tipos en el editor.	pom.xml, META-INF/RASCAL. MF, Plugin.rsc
4. Anotaciones de tipo	La Parte 2 ya tenía val x: T, parámetros x: T, retornos: T y objetos genéricos object List[+A](val item: A). Faltaba el literal de carácter: se agregó 'a' (CharLit / charLit). Tipos básicos: Int, Float, Boolean, Char, String.	Syntax.rsc, AST.rsc
5. Verificar tipos	Nuevo verificador con TypePal: declaraciones con anotación, condiciones de if/for, operadores, argumentos y retorno de métodos, argumentos de new, herencia como subtipo y genéricos con varianza (con List[+A], List [Dog] es subtipo de List[Animal]).	Checker.rsc (nuevo)
6. Regla nueva	Todo lo que usa un objeto debe existir: el padre de extends, los tipos de los parámetros, los parámetros de tipo, el objeto de new X y cada campo o método accedido con obj.x, obj.m(...) u obj.m = method .... Si no existe, se reporta un error.	Checker.rsc

Cambios a la gramática y al AST

Además del literal de carácter, estos cambios fueron necesarios para que los fragmentos de código del enunciado parsearan:

- if como expresión (ifExpr), para val max = if a > b then a else b. Por eso también los bloques {...} y print/println son expresiones. Se eliminaron el no-terminal Body y el constructor ifStmt; el cuerpo de un for es ahora un Statement.
- Método sin cuerpo Obj.m = method(...): T (methodDecl), para métodos abstractos.
- Interpolación corta "$nombre", además de "${expresión}".
- Restricción !>> "{" en object, new y Obj.m = method(...), para que una llave siguiente se lea como el cuerpo y no como un bloque aparte.

En el verificador y el intérprete también se agregó: inferencia de los argumentos de tipo en new (new List (new Dog) es List [Dog]), métodos agregados a un objeto con Obj.m = method ... (los heredan sus hijos e instancias), getClass y llamadas a métodos sin parámetros sin paréntesis.

Pruebas y ejemplos

- Tests.rsc: de 23 a 66 pruebas (tipos, regla nueva, fragmentos del enunciado y resta). Todas pasan.
- Nuevos ejemplos en examples/: ejemplo2.ol (programa correcto), error_tipos.ol (9 errores de tipos), error_objeto.ol (7 violaciones de la regla nueva) y snippet1.ol a snippet4.ol (los fragmentos del enunciado, que se ejecutan).

Problemas de la Parte 2

Problema en la Parte 2	Estado
val x = 1 - 2 era ambiguo: podía leerse como una resta o como val x = 1 seguida de la sentencia -2. El programa no se podía ejecutar.	**Corregido.** Cuando una lista de sentencias tiene varias lecturas, el parser elige la que tiene menos sentencias, es decir, la resta. Una sentencia aparte que empiece con - se escribe entre paréntesis: (-2).